Contents

1 Basic

1.1 Default

```cpp
#include<bits/stdc++.h>
using namespace std;
#define int long long
#define Harry55 ios_base::sync_with_stdio(0), cin.tie
    (0), cin.exceptions(cin.failbit);
#define pii pair<int, int>
#define vi vector<int>
#define ff first
#define ss second
#define all(x) (x).begin(), (x).end()

void solve(){

}

signed main(){
    int t = 1;
    //cin >> t;
    while(t--){
        solve();
    }
}
```

1.2 builtin

```cpp
__builtin_popcountll //二進位有幾個1(記得這是long long)
__builtin_clzll //左起第一個1之前0的個數
__builtin_parityll //1的個數的奇偶性
__builtin_mul_overflow(a,b,&h) //a*b是否溢位,h = a * b
__builtin_add_overflow(a,b,&h)
```

1.3 random

```cpp
mt19937 mt(chrono::steady_clock::now().time_since_epoch
    ().count());
//mt19937_64 mt() -> return randnum
int randint(int l, int r){
    uniform_int_distribution<> dis(l, r); return dis(mt
        );
}
```

1.4 run.bat

```bat
@echo off
g++ ac.cpp -o ac.exe
g++ wa.cpp -o wa.exe
set /a num=1
:loop
    echo %num%
    python gen.py > input
    ac.exe < input > ac
    wa.exe < input > wa
    fc ac wa
    set /a num=num+1
if not errorlevel 1 goto loop
```

1.5 run.sh

```sh
set -e
for ((i=0;;i++))
do
    echo "$i"
    python gen.py > in
    ./ac < in > ac.out
    ./wa < in > wa.out
    diff ac.out wa.out || break
done
```

2 Data Structure

2.1 Segment Tree

```cpp
struct seg_tree{
    #define cl(x) (x << 1)
    #define cr(x) (x << 1) | 1
    vi seg, arr;
    int n;
    void init(int _n, vi &_arr){
        seg = vi(4 * _n + 5);
        arr = _arr;
        n = _n;
        build(1, 0, _n - 1);
    }
    void pull(int id){
        // 修改條件
        seg[id] = min(seg[cl(id)], seg[cr(id)]);
    }
    void build(int id, int l, int r){
        if(l == r){
            seg[id] = arr[l];
            return;
        }
        int mid = (l + r) >> 1;
        build(cl(id), l, mid);
        build(cr(id), mid + 1, r);
        pull(id);
    }
    void upd(int id, int l, int r, int x, int v){
        if(l == r){
            seg[id] = v;
            return;
        }
        int mid = (l + r) >> 1;
        if(x <= mid) upd(cl(id), l, mid, x, v);
        else upd(cr(id), mid + 1, r, x, v);
        pull(id);
    }
    int query(int id, int l, int r, int sl, int sr){
        if(sl <= l && r <= sr){
            return seg[id];
        }
        int mid = (l + r) >> 1, res = INF;
```

```
        if(sl <= mid) res = min(res, query(cl(id), l,
            mid, sl, sr));
        if(mid < sr) res = min(res, query(cr(id), mid +
             1, r, sl, sr));
        return res;
    }
    int find_first(int id, int l, int r, int sl, int sr
        , int k){//二分搜
        if(l > sr || r < sl || seg[id] > k) return -1;
        if(l == r) return l;
        int mid = (l + r) >> 1;
        int res = find_first(cl(id), l, mid, sl, sr, k)
            ;
        if(res == -1) res = find_first(cr(id), mid + 1,
             r, sl, sr, k);
        return res;
    }
    int find_first(int l, int r, int k) {
        return find_first(1, 0, n - 1, l, r, k);
    }
    int query(int sl, int sr){
        return query(1, 0, n - 1, sl, sr);
    }
    void upd(int x, int v){
        upd(1, 0, n - 1, x, v);
    };
};
```

2.2 segment Tree2

```
#include<bits/stdc++.h>
using namespace std;
#define int long long
#define pii pair<int ,int>
#define Harry55 ios::sync_with_stdio(0),cin.tie(0);

struct seg_tree{
    #define cl(x) (x << 1)
    #define cr(x) (x << 1) | 1
    vector<int> seg, arr, tag;
    int n;
    void init(int _n, const vector<int> &_arr){
        seg = vector<int>(4 * _n + 5);
        tag = vector<int>(4 * _n + 5);
        arr = _arr;
        n = _n;
        build(1, 0, _n - 1);
    }
    void pull(int id, int l, int r){
        int mid = (l+r)>>1;
        push(cl(id), l, mid);
        push(cr(id), mid + 1, r);
        seg[id] = seg[cl(id)] + seg[cr(id)];
    }

    void push(int id, int l, int r){
        if(tag[id]) {
            seg[id] += tag[id] * (r - l + 1);
            if(l != r) {
                tag[cl(id)] += tag[id];
                tag[cr(id)] += tag[id];
            }
            tag[id] = 0;
        }
    }

    void build(int id, int l, int r){
        if(l == r){
            seg[id] = arr[l];
            return;
        }
        int mid = (l + r) >> 1;
        build(cl(id), l, mid);
        build(cr(id), mid + 1, r);
        pull(id, l, r);
    }
    void upd(int id, int l, int r, int sl, int sr, int
        v){
        push(id, l, r);
        if(sl <= l && r <= sr){
            tag[id] += v;
            return;
        }
        int mid = (l + r) >> 1;
        if(sl <= mid) upd(cl(id), l, mid, sl, sr, v);
        if(sr > mid) upd(cr(id), mid + 1, r, sl, sr, v)
            ;
        pull(id, l, r);
    }
    int query(int id, int l, int r, int sl, int sr){
        push(id, l, r);
        if(sl <= l && r <= sr){
            return seg[id];
        }
        int mid = (l + r) >> 1, res = 0;
        if(sl <= mid) res += query(cl(id), l, mid, sl,
            sr);
        if(mid < sr) res += query(cr(id), mid + 1, r,
            sl, sr);
        return res;
    }
    int query(int sl, int sr){
        return query(1, 0, n - 1, sl, sr);
    }
    void upd(int sl, int sr, int v) {
        upd(1, 0, n - 1, sl, sr, v);
    }
};
```

2.3 BIT

```
struct BIT{
    int n;
    vector<int> bit;
    #define lowbit(x) (x & -x)
    void init(int _n){
        n = _n + 1;
        bit = vector<int>(n, 0);
    }
    int _query(int x){
        int ans = 0;
        for( ; x > 0 ; x -= lowbit(x)) ans += bit[x];
        return ans;
    }
    void _upd(int x, int v){
        for(; x <= n ; x += lowbit(x)) bit[x] += v;
    }
    void upd(int x, int v) {_upd(x + 1, v);}
    int query(int x) {return _query(x + 1);}
    int query(int l, int r) {return (query(r) - query(l
        - 1));}
};
```

2.4 離散化

```
vi tmp(arr);
sort(all(tmp));
tmp.erase(unique(all(tmp)), tmp.end());
for(int i = 0 ; i < n ; i++) arr[i] = lower_bound(all(
    tmp), arr[i]) - tmp.begin();
```

2.5 DSU

```
struct DSU {
    vector<int> f, sz;
    DSU(int n) : f(n), sz(n) {
        for (int i = 0; i < n; i++) {
            f[i] = i;
            sz[i] = 1;
        }
    }
    int find(int x) {
        if (x == f[x]) return x;
        f[x] = find(f[x]);
        return f[x];
    }
    int merge(int x, int y) {
        x = find(x), y = find(y);
        if (x == y) return 0;
        if (sz[x] < sz[y])
            swap(x, y);
        sz[x] += sz[y];
        f[y] = x;
        return 1
```

```
    }
    bool same(int a, int b) { return (find(a) == find(b
        )); }
};
```

2.6 Kruskal

```
struct Edge {
    int u, v, w;
    bool operator<(const Edge& other) const {return w <
         other.w;}
};
int kruskal(int n, vector<Edge>& edges) {
    dsu.init(n);
    sort(edges.begin(), edges.end());
    int mst_weight = 0;
    int edge_cnt = 0;
    for (auto e : edges) {
        if (dsu.merge(e.u, e.v)) {
            mst_weight += e.w;
            edge_cnt++;
            if (edge_cnt == n - 1) break;
        }
    }
    if (edge_cnt < n - 1) return -1;
    return mst_weight;
}
```

3 Graph

3.1 Topo

```
vector<int> edge[N], deg(N, 0), ans;
bool topo(int n){
    ans.clear();
    queue<int> q;
    for(int i = 1 ; i <= n ; i++) if(deg[i] == 0) q.
        push(i);
    while(!q.empty()){
        int u = q.front(); q.pop();
        ans.push_back(u);
        for(auto i : edge[u]){
            if(--deg[i] == 0) q.push(i);
        }
    }
    return n == ans.size();
}
```

3.2 找環

```
int vis[N] = {0};
bool dfs(int u) {
    if (vis[u] == 1) return true;
    if (vis[u] == 2) return false;
    vis[u] = 1;
    for (auto v : edge[u]) {
        if (dfs(v)) {
            return true;
        }
    }
    vis[u] = 2;
    return false;
}//有向圖找環
bool dfs(int x, int v) {
    if (vis[x]) return true;
    vis[x] = 1;
    for (auto i : edge[x]) {
        if (i == v) continue;
        if (dfs(i, x)) return true;
    }
    return false;
}//無向圖找環
```

3.3 無向圖環路徑

```
int vis[], suc[];
int dfs(int x, int v) { //v -> x 是 tree edge
    if (vis[x]) return x;
    suc[x] = v; //紀錄 x 的父節點 v
    vis[x] = 1;
    for (auto i : edge[x]) {
        if (i == v) continue;
        int tmp = dfs(i, x); // x 的祖先，
        if (tmp) {
            int now = x;
            vector<int> ans;
            ans.push_back(i);
            while (now != tmp) { //找從x到tmp的路徑
                ans.push_back(now);
                now = suc[now];
            }
            ans.push_back(i);
            for (auto v : ans) cout << v << " ";
            cout << "\n";
            exit(0);
        }
    }
    return 0;
}
```

3.4 有向圖環路徑

```
void dfs(int u){
    vis[u] = 1;
    for (auto i : edge[u]) {
        if (vis[i] == 1) {
            vector<int> ans;
            ans.push_back(i);
            int now = u;
            while (now != i) {
                ans.push_back(now);
                now = suc[now];
            }
            ans.push_back(i);
            reverse(ans.begin(), ans.end());
            cout << ans.size() << "\n";
            for (auto i : ans) cout << i << " ";
            exit(0);
        }
        if (vis[i] == 0) {
            suc[i] = u;
            dfs(i);
        }
    }
    vis[u] = 2;
}
```

3.5 無向圖歐拉路徑

```
void dfs(int u, vector<pii> edge[], vector<int> &vis){
    while(!edge[u].empty()){
        auto [v, idx] = edge[u].back(); edge[u].
            pop_back();
        if(vis[idx]) continue;
        vis[idx] = 1;
        dfs(v, edge, vis);
        epath.push_back(idx); // edge path
    }
    path.push_back(u); // node path
}//reverse
```

4 Tree

4.1 is_anc

```
bool is_ancestor(int u,int v){
    return ( in[u] < in[v] && out[v] < out[u] );
}
```

4.2 LCA

```
int getlca(int x, int y) {
    if (is_ancestor(x, y)) return x;
    if (is_ancestor(y, x)) return y;
    for (int i = 19; i >= 0; i--) {
        if (!is_ancestor(anc[x][i], y))
            x = anc[x][i];
    }
    return anc[x][0];
}
```

4.3 倍增

```
int jump[N][logN];
for(int i = 1; i <= log2(N); i++){
    for(int now = 1; now <= N; now++){
        jump[now][i] = jump[jump[now][i - 1]][i - 1];
    }
}
//查詢
int query(int x,int k){
    int i = log2(k);
    while(k){
        if((1<<i) <= k){
            x = jump[x][i];
            k -= (1<<i);
        }
        i--;
    }
    return x;
}
```

5 DP

5.1 LIS

```
vector<int> dp;
for(int i = 0; i < n;++i){
    auto it = lower_bound(all(dp),A[i]) - dp.begin();
    if(it == dp.size()){
        dp.push_back(A[i]);
    }else{
        dp[it] = A[i];
    }
}
cout << dp.size() << '\n';
```

5.2 LCS

```
for(int i = 1; i <= n; ++i){
    for(int j = 1; j <= m; ++j){
        dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
        if(s[i] == t[j]) dp[i][j] = max(dp[i][j],dp[i
            -1][j-1]+1);
    }
}
```

5.3 拆分

```
//目前物品最多拿 k 個
int s = 0;
vector<pair<int,int>> A; // A 是儲存所有捆
for(int r = 1; r * 2 <= k; r *= 2){ // TLE的話先算k最大
    拿幾個
    s += r;
    A.push_back({r*c[i],r*v[i]}); // r個一綑
}
A.push_back({(k-s)*c[i],(k-s)*v[i]}); //最後一捆
```

5.4 編輯距離

```
for(int i = 1; i <= n; ++i)
    for(int j = 1; j <= m; ++j){
    if(s[i-1] == t[j-1]) dp[i][j] = dp[i-1][j-1];
    else {
        dp[i][j] = min({
            dp[i-1][j] + 1,
            dp[i][j-1] + 1,
            dp[i-1][j-1] + 1}
        );
    }
}
```

5.5 SOSDP

```
for(int i = 0;i < n; ++i)
    for (int s = 0; s < (1<<n) ; ++s)
        if (s & (1 << i))
            F[s] += F[s ^ (1 << i)];
//窮舉子集合
for(int s = mask; s ; s = (s-1)&mask;)
```

5.6 p_median

```
for(int h = 1; h <= k;++h){ //窮舉目前放了多少個區間
    for(int i = 1;i <= n;++i){ //第 h 個區間以 pi 為右
        界
        dp[h][i] = inf; //初始化
        for(int j = 1; j <= i; ++j){ //第 h 個區間的左
            界
            dp[h][i] = min(dp[h][i],dp[h-1][j-1] + d[j
                ][i]);
        }
    }
}
cout << dp[k][n];
```

6 search

6.1 binary search

```
int bsearch_1(int l, int r)
{
    while (l < r)
    {
        int mid = (l + r) >> 1;
        if (check(mid)) r = mid;
        else l = mid + 1;
    }
    return l;
}
// ................oooooooooo

int bsearch_2(int l, int r)
{
    while (l < r)
    {
        int mid = (l + r + 1) >> 1;
        if (check(mid)) l = mid;
        else r = mid - 1;
    }
    return l;
}
// oooooooooo..................
```

6.2 三分搜

```
int l = 1,r = 100;
while(l < r) {
    int lmid = l + (r - l) / 3; // l + 1/3區间大小
    int rmid = r - (r - l) / 3;  // r - 1/3區间大小
    lans = cal(lmid),rans = cal(rmid);
    // 求凹函區的极小值
    if(lans <= rans) r = rmid - 1;
    else l = lmid + 1;
}
```

7 shortest path

7.1 Dijkstra

```
const int N = 1e5 + 5;
const int INF = 1e18;
vector<pii> edge[N];
vector<int> dis(N, INF);
void dijk(int s){
    dis[s] = 0;
    vector<int> vis(N, 0);
    priority_queue<pii,vector<pii>,greater<pii>> pq;
    pq.push({0, s});
    while(!pq.empty()){
        int u = pq.top().ss; pq.pop();
        if(vis[u]) continue; vis[u] = 1;
        for(auto [v, w] : edge[u]){
            if(dis[u] + w < dis[v]) {
                dis[v] = dis[u] + w;
                pq.push({dis[v], v});
            }
        }
    }
}
```

7.2 SPFA

```
const int N = 1e5 + 5;
const int INF = 1e18;
vector<pii> edge[N];
vector<int> dis(N, INF);
bool spfa(int s, int n){
    vector<int> cnt(N, 0), vis(N, 0);
    queue<int> Q;
    dis[s] = 0; Q.push(s); vis[s] = 1;
    while (Q.size()) {
        int u = Q.front(); Q.pop();
        vis[u] = 0;
        for (auto [v, w] : edge[u]) {
            if (dis[v] > dis[u] + w) {
                dis[v] = dis[u] + w;
                cnt[v] = cnt[u] + 1;
                if (cnt[v] >= n) {
                    return 1;
                }
                if (!vis[v]) Q.push(v), vis[v] = 1;
            }
        }
    }
    return 0;
}
```

7.3 01BFS

```
void bfs_01(int s, int n) {
    for(int i = 1; i <= n; i++) dis[i] = INF;
    deque<int> dq;
    dis[s] = 0;
    dq.push_front(s);
    while(!dq.empty()) {
        int u = dq.front();dq.pop_front();
        for(auto [v, w] : edge[u]) {
            if (dis[u] + w < dis[v]) {
                dis[v] = dis[u] + w;
                if (w == 0) dq.push_front(v);
                else dq.push_back(v);
            }
        }
    }
}
```

8 Math

8.1 質數篩

```
const int MXN = 1e8 + 50;
const int SQRTMXN = 1e4 + 50;
bitset<MXN> isprime;
void sieve() {
    isprime[1] = 1;
    for (int i = 2; i <= SQRTMXN; i++) {
        if (!isprime[i])
            for (int j = i * i; j < MXN; j += i)
                isprime[j] = 1;
    }
}
```

8.2 fpow

```
int fpow(int a, int b){
    int ret = 1;
    for(;b; b>>=1, a=a*a%MOD) if(b&1) ret = ret*a%MOD;
    return ret;
}
```

8.3 exgcd

```
int exgcd(int a, int b, int &x, int &y) {
    if (b == 0) return x = 1, y = 0, a;
    int g = exgcd(b, a % b, y, x);
    y -= a / b * x;
    return g;
} // ax + by = gcd(a, b)
```

8.4 組合數

```
const int MXN = 1e6 + 5;
int fac[MXN], inv[MXN];
fac[0] = 1;
for(int i = 1 ; i < MXN ; i++) fac[i] = fac[i - 1] * i
    % MOD;
inv[MXN - 1] = fpow(fac[MXN - 1], MOD - 2);
for(int i = N - 2 ; i >= 0 ; i--) inv[i] = inv[i + 1] *
     (i + 1) % MOD;
int C(int n, int k){return fac[n] * inv[k] % MOD * inv[
    n - k] % MOD;}
```

8.5 josephus

```
int josephus(int n, int m){ //n人每m次
    int ans = 0;
    for (int i=1; i<=n; ++i)
        ans = (ans + m) % i;
    return ans;
}
```

9 String

9.1 KMP

```
//pi[i]=最大的k使得s[0...(k-1)] = s[i-(k-1)...i]
vector<int> prefunc(const string& s){
  int n = s.size();
  vector<int> pi(n);
  for(int i=1,j=0;i<n;++i){
    j = pi[i-1];
    while(j && s[j] != s[i]) j = pi[j-1];//取次小LCP
    if(s[j] == s[i]) ++j;
    pi[i] = j;
  }
  return pi;
}
//找s在str中出現的所有位子
vector<int> kmp(string str, string s) {
    vector<int> nxt = prefunc(s);
    vector<int> ans;
    for (int i = 0, j = 0; i < str.size(); i++) {
        while (j && str[i] != s[j]) j = nxt[j - 1];
        if (str[i] == s[j]) j++;
        if (j == s.size()) {
            ans.push_back(i - s.size() + 1);
            j = nxt[j - 1];
        }
    }
    return ans;
}
```